

Лекция для слушателей

Аналитика данных в бизнесе и других сферах. Типы и источники данных

1. Вступление

Здравствуйте, коллеги.

Сегодня мы начинаем практическое знакомство с аналитикой данных через Python и Jupyter Notebook. Наша задача — не просто посмотреть на код, а понять, как с помощью кода можно работать с данными: хранить их, обрабатывать, сравнивать, группировать, визуализировать и делать простые аналитические выводы.

Для нас Python сегодня — это не язык «ради программирования». Это инструмент аналитика. С его помощью можно:

- быстро считать показатели;
- проверять условия;
- обрабатывать списки значений;
- работать с текстовыми данными;
- собирать данные в таблицы;
- строить простые графики;
- готовить основу для дальнейшего анализа.

В бизнесе аналитик часто работает с данными о продажах, клиентах, заказах, товарах, городах, каналах продаж, статусах заявок и финансовых показателях. Поэтому все примеры сегодня будем связывать не с абстрактными переменными, а с понятными бизнес-ситуациями.

2. Что такое данные

Прежде чем писать код, важно договориться о терминах.

Данные — это зафиксированная информация, которую можно хранить, передавать, обрабатывать и анализировать.

Примеры данных:

- сумма заказа;
- дата покупки;
- город клиента;
- категория товара;
- количество проданных единиц;
- email клиента;
- статус заявки;

- оценка пользователя;
- источник перехода на сайт.

На первый взгляд это просто отдельные значения. Но когда таких значений становится много, из них можно получить полезные ответы:

- какой город приносит больше выручки;
- какие товары продаются лучше;
- какие клиенты покупают чаще;
- где продажи ниже плана;
- какой канал продвижения эффективнее;
- как меняется показатель по месяцам.

Аналитика данных начинается с простого вопроса:

что мы хотим узнать на основе имеющихся данных?

3. Типы данных в аналитике

Данные бывают разными. В Python и в аналитике важно понимать, с каким типом данных мы работаем.

Основные типы, которые нам понадобятся:

1. Числа

Например: цена, количество, выручка, скидка, возраст, рейтинг.

2. Строки

Например: имя клиента, город, email, категория товара, статус заказа.

3. Логические значения

Это значения `True` или `False`.

Например: заказ оплачен или не оплачен, клиент активен или неактивен, план выполнен или не выполнен.

4. Списки

Это наборы значений.

Например: список продаж за неделю или список городов.

5. Словари

Это структура «ключ — значение».

Например: карточка клиента, где есть имя, город, сумма покупок и статус.

6. Таблицы

В аналитике чаще всего данные представлены в виде таблицы: строки и столбцы.

В Python для этого мы будем использовать `pandas.DataFrame`.

4. Источники данных

Источник данных — это место, откуда аналитик получает информацию.

Типичные источники данных:

- Excel-файлы;
- CSV-файлы;
- JSON-файлы;
- базы данных;
- CRM-системы;
- сайты;
- API;
- выгрузки из BI-систем;
- логи приложений;
- формы обратной связи;
- ручные таблицы сотрудников.

В реальной работе данные редко приходят в идеальном виде. Часто они требуют подготовки:

- привести названия к одному формату;
- убрать лишние пробелы;
- преобразовать строки в числа;
- проверить пропуски;
- объединить таблицы;
- выделить нужные признаки;
- посчитать итоговые показатели.

Сегодня мы начнем с базовых инструментов, которые позволяют делать эти операции постепенно.

5. Как мы будем работать в Jupyter Notebook

Работать будем в формате Jupyter Notebook или Google Colab.

Ноутбук состоит из ячеек двух основных типов:

1. Markdown-ячейки

В них пишется текст: объяснения, заголовки, выводы, инструкции.

2. Code-ячейки

В них пишется Python-код, который можно запускать и сразу видеть результат.

Важно понимать: ноутбук — это не просто файл с кодом. Это учебная и аналитическая среда, где рядом находятся:

- объяснение;
- код;
- результат выполнения;

- таблицы;
- графики;
- выводы.

Поэтому в аналитике ноутбук часто используется как рабочая тетрадь: в нем можно исследовать данные, фиксировать промежуточные выводы и показывать результат другим людям.

6. Python как калькулятор

Начнем с самого простого.

Python умеет выполнять арифметические операции:

```
2 + 3
10 - 4
5 * 6
20 / 4
```

Для аналитика это базовый уровень: посчитать сумму, разницу, произведение, отношение.

Например, у нас есть цена товара и количество проданных единиц:

```
price = 1500
quantity = 7

revenue = price * quantity
print(revenue)
```

Здесь мы считаем выручку.

Выручка — это один из самых простых и важных бизнес-показателей:

```
выручка = цена * количество
```

Даже такой простой пример уже показывает связь Python с аналитикой. Мы не просто умножаем числа, мы считаем показатель, который нужен бизнесу.

7. Переменные

Переменная — это имя, под которым мы сохраняем значение.

Например:

```
price = 1500
quantity = 7
```

Здесь `price` — это цена, а `quantity` — количество.

Переменные нужны, чтобы:

- не писать одно и то же значение много раз;
- давать данным понятные имена;
- использовать значения в расчетах;
- сохранять промежуточные результаты.

Плохой пример:

```
a = 1500
b = 7
c = a * b
```

Технически код работает, но непонятно, что такое `a`, `b` и `c`.

Лучший пример:

```
price = 1500
quantity = 7
revenue = price * quantity
```

Такой код проще читать. Даже человек, который не писал этот код, понимает смысл расчета.

Для аналитика читаемость кода очень важна. Аналитический код часто нужно передавать коллегам, объяснять руководителю или возвращаться к нему через несколько недель.

8. Типы данных в Python

Python различает типы данных.

Пример целого числа:

```
orders_count = 25
```

Пример дробного числа:

```
average_check = 1340.75
```

Пример строки:

```
city = "Москва"
```

Пример логического значения:

```
is_paid = True
```

Чтобы проверить тип значения, можно использовать функцию `type()`:

```
type(orders_count)
type(average_check)
type(city)
type(is_paid)
```

Почему это важно?

Потому что разные типы данных ведут себя по-разному.

Например, число можно умножить на число:

```
1500 * 7
```

А строка с числом — это уже не число:

```
"1500" * 7
```

Результат будет не математическим умножением, а повторением строки.

Это частая ошибка начинающих аналитиков: данные выглядят как числа, но на самом деле хранятся как текст.

9. Ввод и вывод данных

Для вывода результата используется `print()`:

```
revenue = 10500
print(revenue)
```

Можно выводить не только значение, но и пояснение:

```
print("Выручка:", revenue)
```

Это полезно, когда в ноутбуке много расчетов. Подпись помогает понять, что именно мы вывели.

Функция `input()` позволяет получить значение от пользователя:

```
name = input("Введите имя: ")
```

Но важно помнить: `input()` всегда возвращает строку. Даже если пользователь ввел число, Python сначала воспринимает его как текст.

Поэтому для числовых расчетов нужно преобразование:

```
price = int(input("Введите цену: "))
```

или

```
price = float(input("Введите цену: "))
```

В учебных ноутбуках мы часто используем заранее заданные значения вместо `input()`, чтобы файл можно было выполнить целиком без остановок.

10. Сравнения и условия

Аналитика часто связана с проверками.

Например:

- выполнен ли план продаж;
- больше ли выручка заданного порога;
- является ли клиент активным;
- относится ли заказ к крупным;
- есть ли скидка.

В Python для этого используются сравнения:

```
revenue > 10000  
revenue == 10000  
revenue < 10000
```

Результатом сравнения будет логическое значение:

True

или

False

Условия позволяют выполнить разный код в зависимости от ситуации:

```
revenue = 12000

if revenue >= 10000:
    print("План выполнен")
else:
    print("План не выполнен")
```

Если условий несколько, используется `elif`:

```
revenue = 7500

if revenue >= 10000:
    print("Высокая выручка")
elif revenue >= 5000:
    print("Средняя выручка")
else:
    print("Низкая выручка")
```

Это уже похоже на простую классификацию данных.

Например, можно классифицировать клиента по сумме покупок:

```
total_spent = 45000

if total_spent >= 100000:
    segment = "VIP"
elif total_spent >= 30000:
    segment = "Постоянный клиент"
else:
    segment = "Новый или редкий клиент"

print(segment)
```


11. Строки

Строки используются для текстовых данных.

Примеры строк:

```
client_name = "Иван"  
city = "Москва"  
email = "client@example.com"  
status = "paid"
```

Строки можно объединять:

```
first_name = "Иван"  
last_name = "Петров"  
  
full_name = first_name + " " + last_name  
print(full_name)
```

Можно менять регистр:

```
city = "москва"  
print(city.upper())
```

Можно проверять содержимое:

```
email = "client@example.com"  
  
if "@" in email:  
    print("Похоже на email")  
else:  
    print("Некорректный email")
```

В аналитике строки встречаются постоянно:

- имена;
- города;
- категории;
- статусы;
- email;
- комментарии;
- названия товаров.

Часто текстовые данные нужно приводить к единому виду. Например, если в таблице один и тот же город записан как `Москва`, `москва`, `МОСКВА`, то для анализа это может стать проблемой.

12. Списки

Список — это структура, которая хранит несколько значений.

Пример:

```
sales = [1200, 3400, 2800, 4100]
```

Это может быть список продаж за несколько дней.

Можно получить отдельный элемент списка по индексу:

```
sales[0]
```

Важно: индексация в Python начинается с нуля.

Первый элемент имеет индекс `0`, второй — `1`, третий — `2`.

Можно изменить элемент списка:

```
sales[0] = 1500
```

Списки полезны, когда у нас есть набор значений, с которыми нужно работать последовательно.

Например, посчитаем общую сумму продаж:

```
sales = [1200, 3400, 2800, 4100]

total_sales = sum(sales)
print(total_sales)
```

А теперь среднее значение:

```
average_sales = sum(sales) / len(sales)
print(average_sales)
```

13. Цикл for

Цикл `for` позволяет пройти по каждому элементу списка.

Пример:

```
sales = [1200, 3400, 2800, 4100]

for sale in sales:
    print(sale)
```

Здесь Python берет каждое значение из списка и выполняет действие.

Можно не просто печатать значения, а выполнять расчет:

```
sales = [1200, 3400, 2800, 4100]

for sale in sales:
    increased_sale = sale * 1.1
    print(increased_sale)
```

Например, мы увеличили каждую продажу на 10%.

Циклы нужны, когда действие нужно повторить много раз:

- проверить все заказы;
- обработать всех клиентов;
- пройти по всем городам;
- посчитать показатели по списку значений.

14. Накопление результата в цикле

Частый аналитический прием — пройти по списку и накопить результат.

Пример:

```
sales = [1200, 3400, 2800, 4100]

total = 0

for sale in sales:
    total = total + sale

print(total)
```

Что происходит:

1. Создаем переменную `total` со значением `0`.
2. Берем первую продажу и добавляем к `total`.
3. Берем вторую продажу и добавляем к `total`.
4. Продолжаем до конца списка.

5. Получаем итоговую сумму.

Такой принцип лежит в основе многих аналитических расчетов.

15. Кортежи

Кортеж похож на список, но его нельзя изменять.

Пример:

```
client = ("Иван", "Москва", 25000)
```

Если список можно изменить, то кортеж обычно используют для фиксированных наборов данных.

Например, координаты, дата рождения, фиксированная пара значений.

Попытка изменить элемент кортежа приведет к ошибке. Это нормально: Python защищает данные от изменения.

Для начинающего аналитика важно запомнить:

- список изменяемый;
 - кортеж неизменяемый.
-

16. Множества

Множество хранит только уникальные значения.

Пример:

```
cities = ["Москва", "Казань", "Москва", "Самара", "Казань"]  
  
unique_cities = set(cities)  
print(unique_cities)
```

Это очень полезно в аналитике.

Например, у нас есть список городов по заказам. Один город может встречаться много раз. Но если нужно узнать, из каких уникальных городов были заказы, множество решает эту задачу быстро.

Множества помогают:

- убрать дубли;

- получить уникальные значения;
- сравнивать наборы данных.

17. Словари

Словарь хранит данные в формате «ключ — значение».

Пример:

```
client = {  
    "name": "Иван",  
    "city": "Москва",  
    "total_spent": 45000  
}
```

Чтобы получить значение, нужно обратиться по ключу:

```
client["city"]
```

Словарь удобно использовать, когда нужно описать один объект.

Например, клиент:

```
client = {  
    "name": "Анна",  
    "city": "Казань",  
    "orders": 5,  
    "total_spent": 32000  
}
```

Заказ:

```
order = {  
    "order_id": 101,  
    "city": "Москва",  
    "amount": 7500,  
    "status": "paid"  
}
```

Товар:

```
product = {  
    "name": "Ноутбук",  
    "category": "Электроника",  
}
```

```
}    "price": 85000
```

Словари часто становятся промежуточным шагом перед созданием таблицы.

18. Управление циклом: break и continue

Иногда цикл нужно остановить досрочно.

Для этого используется `break`.

Пример:

```
sales = [1200, 3400, 2800, 15000, 4100]

for sale in sales:
    if sale > 10000:
        print("Найден крупный заказ:", sale)
        break
```

Здесь мы ищем первый крупный заказ. Как только нашли, цикл можно остановить.

Команда `continue` работает иначе. Она не останавливает весь цикл, а пропускает текущий шаг.

Пример:

```
orders = [1200, 0, 3400, 0, 2800]

for order in orders:
    if order == 0:
        continue
    print("Обрабатываем заказ:", order)
```

Здесь нулевые заказы пропускаются, а остальные обрабатываются.

Разница:

- `break` — полностью остановить цикл;
- `continue` — пропустить текущий элемент и перейти к следующему.

19. Генераторы списков

Генератор списка, или list comprehension, — это компактная запись цикла.

Обычный вариант:

```
sales = [1200, 3400, 2800]
new_sales = []

for sale in sales:
    new_sales.append(sale * 1.1)

print(new_sales)
```

Короткий вариант:

```
new_sales = [sale * 1.1 for sale in sales]
print(new_sales)
```

Такой синтаксис удобен, когда нужно быстро преобразовать список.

Пример: оставить только крупные продажи:

```
sales = [1200, 3400, 15000, 2800, 22000]

large_sales = [sale for sale in sales if sale > 10000]
print(large_sales)
```

Для начинающих важно сначала понять обычный цикл, а уже потом использовать короткую запись.

20. Функции

Функция — это именованный блок кода, который можно использовать повторно.

Пример:

```
def calculate_revenue(price, quantity):
    return price * quantity
```

Теперь можно вызвать функцию:

```
result = calculate_revenue(1500, 7)
print(result)
```

Функции нужны, когда один и тот же расчет используется несколько раз.

Например, расчет выручки:

```
def calculate_revenue(price, quantity):  
    return price * quantity
```

Расчет среднего чека:

```
def calculate_average_check(revenue, orders_count):  
    return revenue / orders_count
```

Проверка крупного заказа:

```
def is_large_order(amount):  
    return amount >= 10000
```

Функции делают код:

- короче;
- понятнее;
- удобнее для повторного использования;
- менее подверженным ошибкам.

21. print() и return

Это одна из самых важных тем.

`print()` выводит значение на экран.

`return` возвращает значение из функции, чтобы его можно было использовать дальше.

Пример с `print()`:

```
def show_revenue(price, quantity):  
    print(price * quantity)
```

Эта функция показывает результат, но не возвращает его для дальнейших расчетов.

Пример с `return`:

```
def calculate_revenue(price, quantity):  
    return price * quantity
```

Теперь результат можно сохранить:


```
revenue = calculate_revenue(1500, 7)
tax = revenue * 0.2

print(tax)
```

Для аналитика `return` особенно важен, потому что расчеты часто связаны между собой. Один результат становится входом для следующего расчета.

22. Переход к таблицам

До этого мы работали с отдельными значениями, списками и словарями.

Но в аналитике чаще всего данные представлены в виде таблицы.

Пример таблицы продаж:

Месяц	Город	Категория	Продажи
Январь	Москва	A	1200
Февраль	Казань	B	3400
Март	Москва	A	2800

В Python для работы с таблицами часто используется библиотека `pandas`.

Обычно ее подключают так:

```
import pandas as pd
```

23. DataFrame

`DataFrame` — это таблица в `pandas`.

Создадим данные в виде словаря:

```
data = {
    "Месяц": ["Январь", "Февраль", "Март"],
    "Продажи": [1200, 3400, 2800]
}
```

Преобразуем словарь в таблицу:

```
df = pd.DataFrame(data)
df
```

В результате получим таблицу.

Для аналитика это важный момент:
словарь Python может стать основой для табличной структуры.

В DataFrame есть:

- строки;
- столбцы;
- индекс;
- значения.

Можно выбрать один столбец:

```
df["Продажи"]
```

Можно посмотреть первые строки:

```
df.head()
```

Можно посмотреть информацию о таблице:

```
df.info()
```

Можно получить описательную статистику:

```
df.describe()
```

24. Описательная статистика

Описательная статистика помогает быстро понять числовые данные.

Например:

```
df["Продажи"].describe()
```

Мы можем увидеть:

- количество значений;

- среднее;
- минимум;
- максимум;
- квартильные значения.

Для начинающего аналитика это первый способ быстро осмотреть данные.

Если максимальное значение сильно больше остальных, возможно, есть выброс.

Если среднее сильно отличается от медианного уровня, данные могут быть неравномерными.

Если количество значений меньше ожидаемого, возможно, есть пропуски.

25. Группировка данных

Одна из самых частых задач аналитика — сгруппировать данные.

Например:

- продажи по городам;
- выручка по категориям;
- количество заказов по статусам;
- средний чек по каналам продаж.

Пример:

```
sales_by_city = df.groupby("Город")["Продажи"].sum()  
print(sales_by_city)
```

Логика группировки:

1. Берем таблицу.
2. Делим строки на группы по выбранному столбцу.
3. Для каждой группы считаем показатель.
4. Получаем итоговую таблицу.

Это очень похоже на вопрос:

Сколько продаж было в каждом городе?

Или:

Какая категория принесла больше выручки?

26. Сводные таблицы

Сводная таблица нужна, когда мы хотим посмотреть данные сразу по двум измерениям.

Например:

- строки — города;
- столбцы — категории;
- значения — сумма продаж.

Пример:

```
pivot = df.pivot_table(  
    values="Продажи",  
    index="Город",  
    columns="Категория",  
    aggfunc="sum"  
)  
  
pivot
```

Такую таблицу удобно читать:

- по строкам видим города;
- по столбцам видим категории;
- внутри таблицы видим показатель.

Это похоже на сводные таблицы в Excel, только создается с помощью Python.

27. Визуализация данных

Таблица дает точные значения, но график помогает быстрее увидеть картину.

Например, если у нас есть продажи по месяцам, можно построить линейный график:

```
import matplotlib.pyplot as plt  
  
plt.plot(df["Месяц"], df["Продажи"])  
plt.title("Продажи по месяцам")  
plt.xlabel("Месяц")  
plt.ylabel("Продажи")  
plt.show()
```

График помогает увидеть:

- рост;
- снижение;
- резкие изменения;
- сезонность;
- необычные значения.

Для сравнения категорий удобно использовать столбчатую диаграмму:

```
sales_by_city.plot(kind="bar")
plt.title("Продажи по городам")
plt.xlabel("Город")
plt.ylabel("Продажи")
plt.show()
```

Столбчатая диаграмма хорошо подходит, когда нужно сравнить категории между собой: города, товары, каналы, статусы.

28. Мини-проект: первичный анализ продаж

Теперь объединим изученные темы в маленький аналитический сценарий.

Представим, что у нас есть таблица заказов.

```
orders = pd.DataFrame({
    "order_id": [1, 2, 3, 4, 5, 6],
    "city": ["Москва", "Казань", "Москва", "Самара", "Казань", "Москва"],
    "category": ["A", "B", "A", "C", "B", "C"],
    "amount": [12000, 7500, 18000, 5400, 9200, 15000],
    "status": ["paid", "paid", "paid", "cancelled", "paid", "paid"]
})
```

Первый шаг — посмотреть данные:

```
orders.head()
```

Второй шаг — посчитать базовые KPI:

```
total_revenue = orders["amount"].sum()
average_check = orders["amount"].mean()
orders_count = len(orders)

print("Общая выручка:", total_revenue)
print("Средний чек:", average_check)
print("Количество заказов:", orders_count)
```

Третий шаг — посмотреть выручку по городам:

```
revenue_by_city = orders.groupby("city")["amount"].sum()
print(revenue_by_city)
```

Четвертый шаг — построить сводную таблицу:

```
pivot = orders.pivot_table(  
    values="amount",  
    index="city",  
    columns="category",  
    aggfunc="sum"  
)  
  
pivot
```

Пятый шаг — построить график:

```
revenue_by_city.plot(kind="bar")  
plt.title("Выручка по городам")  
plt.xlabel("Город")  
plt.ylabel("Выручка")  
plt.show()
```

Шестой шаг — сделать простой вывод:

```
best_city = revenue_by_city.idxmax()  
best_revenue = revenue_by_city.max()  
  
print("Город с максимальной выручкой:", best_city)  
print("Выручка:", best_revenue)
```

Это уже похоже на настоящий аналитический процесс.

Мы:

1. получили данные;
2. посмотрели структуру;
3. посчитали показатели;
4. сгруппировали данные;
5. построили сводную таблицу;
6. визуализировали результат;
7. сделали вывод.

29. Как думать как аналитик

Когда вы работаете с данными, важно не просто запускать код, а задавать вопросы.

Перед анализом:

- Какие данные у нас есть?
- Что означает каждый столбец?
- Какие типы данных используются?

- Есть ли числовые показатели?
- Есть ли категории?
- Что мы хотим узнать?

Во время анализа:

- Какие значения повторяются?
- Где максимум и минимум?
- Какие группы отличаются?
- Есть ли странные значения?
- Все ли данные корректны?

После анализа:

- Какой главный вывод?
- Что можно показать в таблице?
- Что лучше показать на графике?
- Что может быть полезно бизнесу?
- Какие данные нужны дополнительно?

Код — это инструмент.

Аналитическая ценность появляется тогда, когда мы превращаем данные в понятный вывод.

30. Частые ошибки начинающих

Ошибка 1. Путать строку и число.

```
"1000" + "500"
```

Результат будет:

```
"1000500"
```

А не `1500`.

Ошибка 2. Забывать, что индексация начинается с нуля.

```
sales = [100, 200, 300]  
sales[0]
```

Первый элемент — это `sales[0]`.

Ошибка 3. Использовать переменную до создания.

```
print(revenue)
```

Если переменная `revenue` еще не создана, Python выдаст ошибку.

Ошибка 4. Путать `print()` и `return`.

`print()` показывает результат.

`return` возвращает результат из функции.

Ошибка 5. Забывать про отступы.

В Python отступы являются частью синтаксиса:

```
if revenue > 10000:  
    print("Высокая выручка")
```

Ошибка 6. Не проверять данные перед анализом.

Перед расчетами полезно выполнить:

```
df.head()  
df.info()  
df.describe()
```

31. Итог занятия

Сегодня мы прошли путь от простых команд Python до первичного анализа табличных данных.

Мы разобрали:

- что такое данные;
- какие бывают типы данных;
- какие бывают источники данных;
- как работать в Jupyter Notebook;
- как использовать переменные;
- как выполнять расчеты;
- как проверять условия;
- как работать со строками;
- как использовать списки, кортежи, множества и словари;
- как применять циклы;
- зачем нужны функции;
- что такое `DataFrame` ;
- как группировать данные;
- как строить сводные таблицы;
- как визуализировать результаты.

Главная идея занятия:

Python помогает аналитику автоматизировать работу с данными: считать, проверять, группировать, визуализировать и делать выводы.

На следующем этапе мы будем глубже работать с источниками данных, загрузкой файлов, обработкой таблиц и подготовкой данных к анализу.

32. Контрольные вопросы для слушателей

1. Что такое данные?
 2. Чем число отличается от строки?
 3. Для чего нужны переменные?
 4. Что делает функция `print()`?
 5. Зачем нужно преобразовывать строку в число?
 6. Чем список отличается от кортежа?
 7. Для чего используется множество?
 8. Что такое словарь?
 9. Чем `break` отличается от `continue`?
 10. Зачем нужны функции?
 11. В чем разница между `print()` и `return`?
 12. Что такое `DataFrame`?
 13. Для чего используется `groupby()`?
 14. Для чего нужна сводная таблица?
 15. Когда лучше использовать столбчатую диаграмму?
-

33. Практическое задание для закрепления

Создайте таблицу заказов со следующими столбцами:

- номер заказа;
- город;
- категория товара;
- сумма заказа;
- статус заказа.

Затем выполните действия:

1. Выведите первые строки таблицы.
2. Посчитайте общую сумму заказов.
3. Посчитайте средний чек.
4. Найдите выручку по городам.
5. Найдите выручку по категориям.
6. Постройте столбчатую диаграмму выручки по городам.
7. Сделайте короткий вывод: какой город принес больше всего выручки.

Пример вывода:

«Наибольшая выручка получена в Москве. Основной вклад внесли заказы категории А. Для дальнейшего анализа стоит посмотреть количество заказов и средний чек по каждому городу».

34. Финальная мысль

Аналитика данных — это не только знание инструментов. Это умение задавать вопросы, аккуратно работать с информацией и объяснять результат понятным языком.

Python, pandas и графики — это техническая часть.

Но главный результат аналитика — это вывод, который помогает принять решение.